BITS Pilani

Machine Learning Operations (MLOps) – AIMLCZG523

End-to-End MLOps Pipeline for Heart Disease Risk Prediction

Assignment 01 – Project Report

Student ID	2025ae05251
Student name	Vadapalli Rohini
Course	MLOps (AIMLCZG523)
Code repository	https://github.com/2025ae05251-LAB/-heart-disease-mlops
Deployed API	Minikube (local Kubernetes) – access instructions in Section 9
Demo video	https://drive.google.com/file/d/1dU582TEm_Sui5CwyYUseRREXGuwQUP6O/view?usp=drive_link

1. Introduction and Objectives

This report documents the design, implementation and deployment of a reproducible machine learning solution that predicts the presence of heart disease from patient health data. The focus of the work is not only model accuracy but the complete MLOps lifecycle: data acquisition, exploratory analysis, model development, experiment tracking, packaging, automated testing, CI/CD, containerization, Kubernetes deployment, and monitoring.

The dataset is the Heart Disease UCI (Cleveland) dataset. The final system exposes a FastAPI service that accepts a JSON patient record at /predict and returns a prediction together with a confidence score. It runs in Docker, is deployed on a local Kubernetes cluster (Minikube) behind a LoadBalancer service, and is monitored with Prometheus and Grafana.

1.1 Technology stack

Area	Tools used
Data & modelling	Python, pandas, NumPy, scikit-learn (Pipeline, ColumnTransformer, GridSearchCV)
Visualisation	matplotlib, seaborn (Jupyter notebook notebooks/EDA.ipynb)
Experiment tracking	MLflow 3.17 with SQLite backend (mlflow.db)
Serving	FastAPI, Uvicorn, Pydantic validation, prometheus-client
Quality & CI/CD	pytest, pytest-cov, flake8, GitHub Actions
Containers & orchestration	Docker, Kubernetes (Minikube), Deployment + Service manifests
Monitoring	Prometheus (ServiceMonitor), Grafana dashboard, file + console logging

1.2 Repository structure

```
src/            download_data.py, preprocess.py, train.py, api.py
notebooks/      EDA.ipynb
data/raw, data/processed   heart.csv, heart_clean.csv
models/         model.joblib, feature_columns.json, model_comparison.csv
tests/          test_data.py, test_model.py, test_api.py
k8s/            deployment.yaml, service.yaml
monitoring/     servicemonitor.yaml, grafana-dashboard.json
scripts/        generate_traffic.py (load generator)
.github/workflows/ci.yml   CI pipeline
Dockerfile, requirements.txt, requirements-api.txt, sample_input.json
screenshots/    evidence for every stage
```

2. Setup and Installation Instructions

All scripts run from a clean environment using the pinned requirements file.

```
git clone https://github.com/2025ae05251-LAB/-heart-disease-mlops.git
cd -heart-disease-mlops
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt

python src/download_data.py     # fetch UCI data -> data/raw/heart.csv
python src/preprocess.py        # clean -> data/processed/heart_clean.csv
python src/train.py             # train, tune, log to MLflow, save model
pytest -v --cov=src             # run unit tests
flake8 src tests                # lint

mlflow ui --backend-store-uri sqlite:///mlflow.db     # http://localhost:5000
uvicorn src.api:app --port 8000                       # run API locally
```

Two requirement files are used: requirements.txt (full development environment including MLflow, seaborn, pytest and flake8) and requirements-api.txt (a minimal runtime set for the serving image, which keeps the Docker image small).

3. Data Acquisition and Exploratory Data Analysis

3.1 Data acquisition

src/download_data.py downloads processed.cleveland.data directly from the UCI repository and assigns the 14 standard column names (age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target). If the direct download fails, the script falls back to the ucimlrepo package (dataset id 45), so the data step remains reproducible. The raw file contains 303 rows and 14 columns.

3.2 Cleaning and preprocessing

src/preprocess.py performs the following steps and writes data/processed/heart_clean.csv:

- **Missing values:** ca has 4 missing values and thal has 2. ca is a count, so it is imputed with the median; thal is categorical, so it is imputed with the mode.
- **Duplicates:** exact duplicate rows are removed (none were present, so all 303 rows are retained).
- **Target binarisation:** the original target is a severity score from 0 to 4. It is converted to binary (0 = no disease, 1 = disease for values > 0), matching the problem statement.
- **Typing:** integer-valued columns are cast to int; oldpeak remains float.

After cleaning there are no missing values, and the classes are 164 healthy (54.1%) versus 139 diseased (45.9%) – a nearly balanced problem, so accuracy is meaningful and no resampling is needed.

3.3 Exploratory analysis

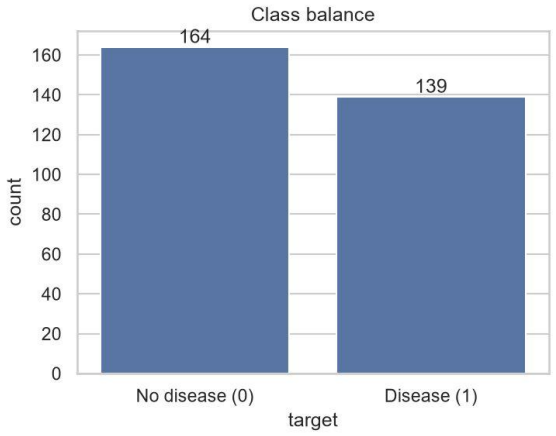


Figure 1: Class balance of the binary target

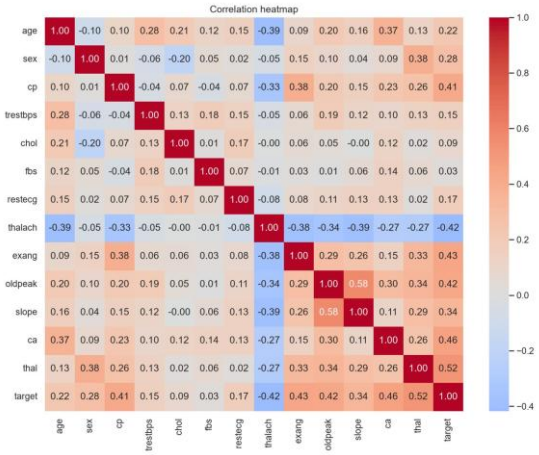


Figure 2: Correlation heatmap

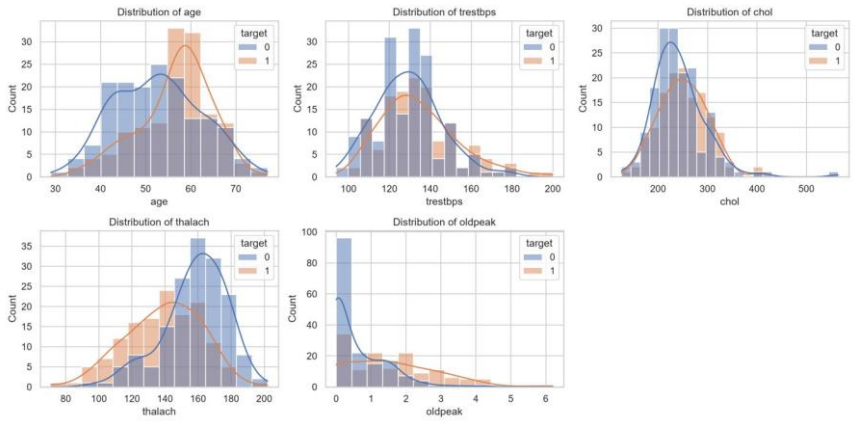


Figure 3: Histograms of the numeric features

The histograms show that age is roughly bell-shaped around the mid-50s, cholesterol is right-skewed with a few extreme values (maximum 564), and oldpeak is heavily concentrated near zero with a long tail. These distributions motivated standardising the numeric features.

In the correlation heatmap, the target correlates positively with ca, oldpeak, exang and cp, and negatively with thalach (maximum heart rate achieved). This agrees with clinical intuition: reduced exercise capacity, exercise-induced angina and more blocked vessels are associated with disease. No pair of predictors is so strongly correlated that it creates serious multicollinearity concerns.

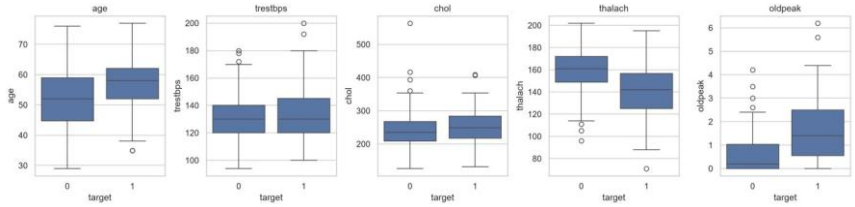


Figure 4: Boxplots of numeric features

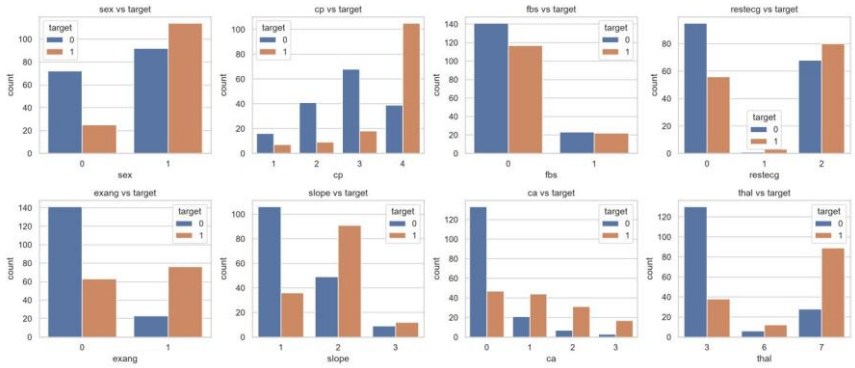


Figure 5: Categorical features versus target

The categorical plots show that chest-pain type, exercise-induced angina, ST slope and thal are strongly discriminative, which is why they are retained and one-hot encoded in the model.

4. Feature Engineering and Model Development

4.1 Feature engineering

All feature engineering lives inside a scikit-learn ColumnTransformer that is part of the model Pipeline. This guarantees that exactly the same transformations are applied at training and at inference time and prevents data leakage during cross-validation.

Feature group	Columns	Transformation
Numeric	age, trestbps, chol, thalach, oldpeak, ca	StandardScaler (zero mean, unit variance)
Nominal categorical	cp, restecg, slope, thal	OneHotEncoder(handle_unknown='ignore')
Binary	sex, fbs, exang	Passed through unchanged

4.2 Models, tuning and validation protocol

The data is split 80/20 into train and hold-out test sets using a stratified split with a fixed random state (42). Two classifiers were trained, each wrapped in the same preprocessing pipeline and tuned with GridSearchCV using stratified 5-fold cross-validation on the training set only and ROC-AUC as the selection metric:

Model	Search grid	Best parameters
Logistic Regression	C ∈ {0.01, 0.1, 1, 10}; penalty = l2	C = 1, penalty = l2
Random Forest	n_estimators ∈ {100, 200}; max_depth ∈ {3, 5, None}; min_samples_split ∈ {2, 5}	n_estimators = 200, max_depth = 3, min_samples_split = 2

Logistic Regression was chosen as an interpretable, well-calibrated baseline. Random Forest was chosen to capture non-linear effects and feature interactions. The test set is touched only once, after tuning, to report unbiased performance.

4.3 Results

Model (split)	Accuracy	Precision	Recall	ROC-AUC
LogReg – 5-fold CV	0.839	0.877	0.766	0.907
RandForest – 5-fold CV	0.822	0.826	0.783	0.897
LogReg – hold-out test	0.869	0.813	0.929	0.958
RandForest – hold-out test	0.885	0.862	0.893	0.948

Both models perform similarly. Random Forest has slightly higher test accuracy and precision, while Logistic Regression has higher recall and higher ROC-AUC in both cross-validation and on the test set. Because the model selection criterion is ROC-AUC, and because in a medical screening setting missing a diseased patient (low recall) is the costlier error, Logistic Regression was selected as the final model (test ROC-AUC 0.958, recall 0.929). Its simplicity also gives faster inference and easier explanation. With only 303 records, the difference between the two models is within the noise one would expect from a small test set (61 patients), which is an honest limitation noted in Section 12.

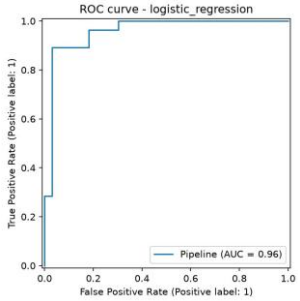


Figure 6: ROC – Logistic Regression

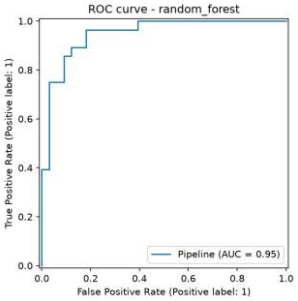


Figure 7: ROC – Random Forest

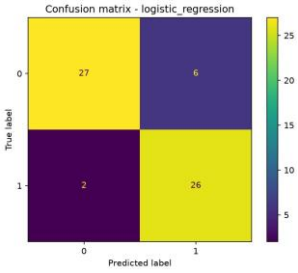


Figure 8: Confusion matrix – Logistic Regression

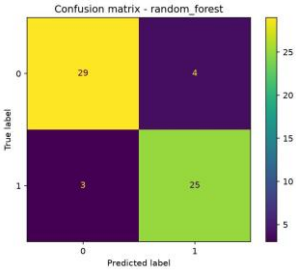


Figure 9: Confusion matrix – Random Forest

5. Experiment Tracking with MLflow

MLflow is configured with a SQLite tracking store (sqlite:///mlflow.db) and a single experiment named heart-disease-classification. Each model is trained inside its own run (run names: logistic_regression and random_forest). For every run the following are logged:

- Parameters: model_type, cv_folds (5), test_size (0.2), and all tuned hyper-parameters (model__C, model__penalty, model__n_estimators, model__max_depth, model__min_samples_split).
- Metrics: cv_accuracy, cv_precision, cv_recall, cv_roc_auc and test_accuracy, test_precision, test_recall, test_roc_auc.
- Artifacts: ROC-curve and confusion-matrix PNGs, plus the complete fitted pipeline logged with mlflow.sklearn.log_model (preprocessing and model together).

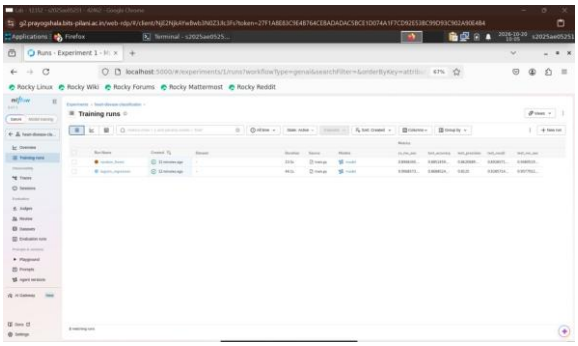


Figure 10: MLflow experiment view listing both runs

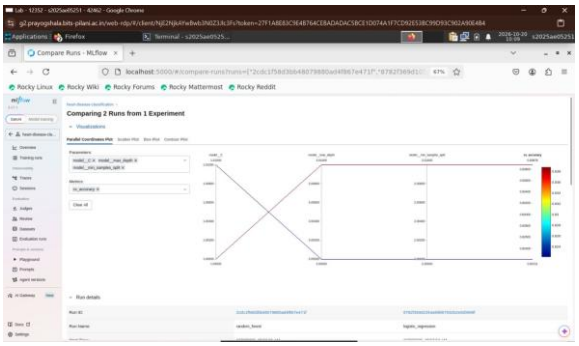


Figure 11: MLflow run comparison (parallel-coordinates plot)

6. Model Packaging and Reproducibility

- **Reusable model format:** the winning pipeline is saved as models/model.joblib. It contains the ColumnTransformer and classifier, so raw patient records can be fed directly to predict_proba. The same pipeline is also stored in MLflow's native model format.
- **Schema file:** models/feature_columns.json fixes the expected column order for inference, and models/model_comparison.csv records the metrics of all candidates.
- **Pinned dependencies:** requirements.txt and requirements-api.txt pin exact package versions (e.g., scikit-learn 1.9.1, pandas 3.0.6, fastapi 0.146.0), so training and serving environments match.
- **Determinism:** a fixed RANDOM_STATE = 42 is used for the data split, cross-validation folds and both estimators.
- **Single source of preprocessing:** because scaling and encoding live inside the pipeline, there is no separate transformer that could drift out of sync with the model.

7. CI/CD Pipeline and Automated Testing

7.1 Unit tests

Fourteen pytest tests are organised in the tests/ folder (pytest.ini sets pythonpath and test path):

File	What is tested
test_data.py	Raw file has the expected 14 columns; cleaned data has no missing values; target is binary {0,1}; no rows or columns lost; age, resting blood pressure and sex values lie in plausible ranges.
test_model.py	Model returns a binary label; probabilities are valid and sum to 1; batch prediction works; saved comparison table shows test ROC-AUC above 0.75.
test_api.py	/health returns 200; /predict returns prediction, label and confidence in [0.5, 1]; missing field and out-of-range age return HTTP 422; /metrics exposes api_requests_total.

7.2 GitHub Actions workflow

The workflow .github/workflows/ci.yml runs on every push and pull request to main (and can be triggered manually). It has two dependent jobs:

1. **build-test-train:** checkout → set up Python 3.14 with pip caching → install requirements → lint with flake8 (src and tests) → download and preprocess data → train models → run pytest with coverage. Any lint error, failing test or script exception fails the job and stops the pipeline, with the full log visible in the run. Training outputs (models, mlflow.db, training plots, coverage.xml, pytest.log) are uploaded as the training-outputs artifact, even on failure.

2. **docker-build-test (needs the first job):** builds the image, starts the container, polls /health until ready, POSTs sample_input.json to /predict and asserts the response contains a valid prediction and a confidence between 0.5 and 1.0. Container logs are always uploaded as the docker-logs artifact.

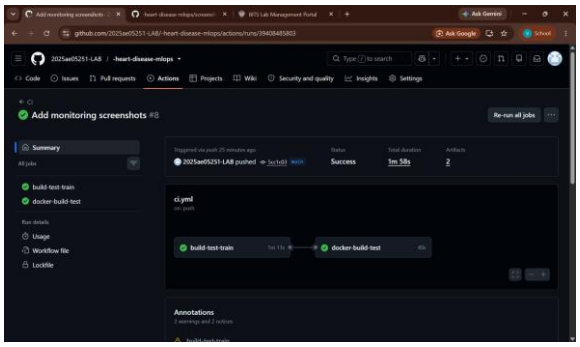


Figure 12: Successful GitHub Actions run (both jobs green, 1m 58s)

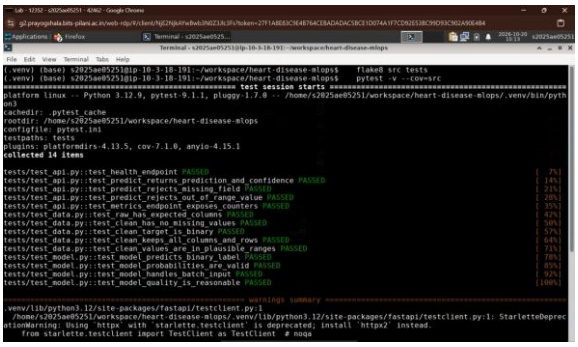


Figure 13: Pytest coverage report

8. Model Containerization

The serving API is implemented in src/api.py with FastAPI. It loads the saved pipeline at start-up and exposes three endpoints:

Endpoint	Method	Purpose
/predict	POST	Accepts a JSON patient record (13 features, validated by a Pydantic model with range checks) and returns prediction (0/1), a text label, confidence and probability_disease.
/health	GET	Liveness/readiness check, used by Kubernetes probes and CI.
/metrics	GET	Prometheus metrics: api_requests_total, api_predictions_total, api_request_latency_seconds.

Confidence is defined as the probability of the predicted class, i.e. max(p, 1 − p), so it is always between 0.5 and 1. Each prediction is also logged (inputs, prediction, confidence and latency in ms) to both the console and logs/api.log.

The Dockerfile uses a slim Python base image, installs only requirements-api.txt, and copies just the API module and the models folder, keeping the image small and the attack surface low. A .dockerignore keeps data, notebooks and screenshots out of the build context.

```
docker build -t heart-api:v1 .
docker run -d -p 8000:8000 --name heart-api heart-api:v1
curl -X POST http://localhost:8000/predict \
     -H "Content-Type: application/json" -d @sample_input.json
```

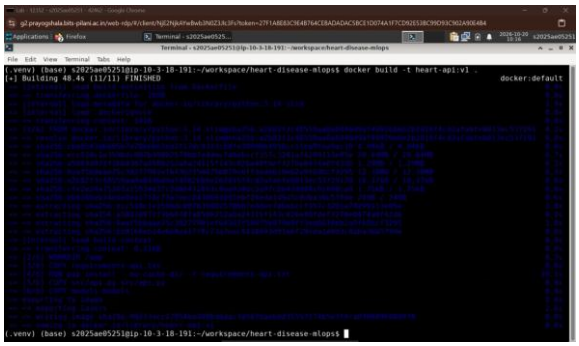


Figure 14: Docker image build

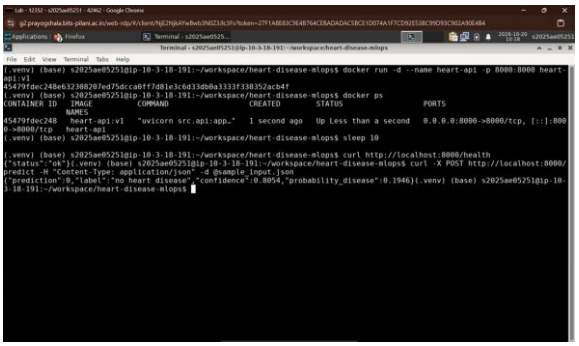


Figure 15: Container running and answering a /predict request with the sample input

9. Production Deployment on Kubernetes

The container was deployed to a local Kubernetes cluster using Minikube. Two manifests in k8s/ define the deployment:

- **deployment.yaml:** 2 replicas of heart-api:v1 (imagePullPolicy: Never because the image is built into Minikube's Docker daemon), container port 8000, readiness probe (/health, 5 s delay) and liveness probe (/health, 10 s delay), and resource requests/limits (100m–500m CPU, 128–512 Mi memory).
- **service.yaml:** a Service of type LoadBalancer mapping port 80 to container port 8000, selecting pods labelled app: heart-api. On Minikube the external IP is provided through minikube tunnel.

```
minikube start
eval $(minikube docker-env)          # build inside the cluster's Docker
docker build -t heart-api:v1 .
kubectl apply -f k8s/
kubectl get pods,svc
minikube tunnel                      # exposes the LoadBalancer on localhost
curl -X POST http://localhost/predict -H "Content-Type: application/json" -d
@sample_input.json
```

Access instructions for local testing: after the steps above the API is reachable at http://localhost/predict (and the interactive Swagger UI at http://localhost/docs).

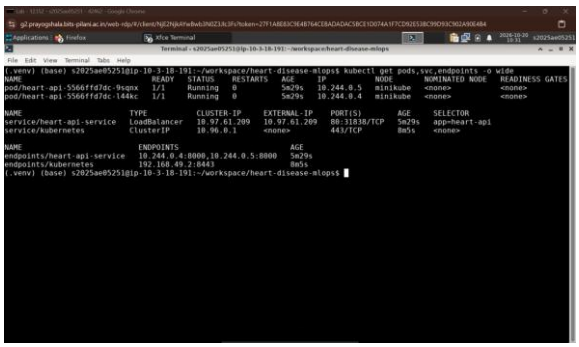


Figure 16: Two running pods

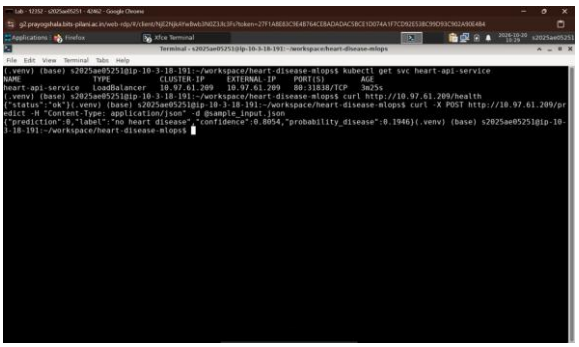


Figure 17: LoadBalancer service and successful /predict call through it

10. Monitoring and Logging

10.1 Logging

Every /predict request is logged with its input record, predicted class, confidence and latency. Logs go to the console (visible through kubectl logs and docker logs) and to logs/api.log. Validation failures are returned as HTTP 422 with clear error messages by FastAPI.

10.2 Metrics with Prometheus and Grafana

The API exposes three Prometheus metrics: a request counter labelled by endpoint and status, a prediction counter labelled by predicted class, and a latency histogram for /predict. A ServiceMonitor (monitoring/servicemonitor.yaml) tells Prometheus to scrape /metrics from the service every 15 seconds. A Grafana dashboard (monitoring/grafana-dashboard.json, exported from the UI) visualises:

requests per second by endpoint, total requests, healthy replicas, predictions by class over 5 minutes, and p95 and average /predict latency.

scripts/generate_traffic.py sends randomised but realistic patient records to the API (300 requests by default, at 5 requests per second) so that the dashboard shows live data.

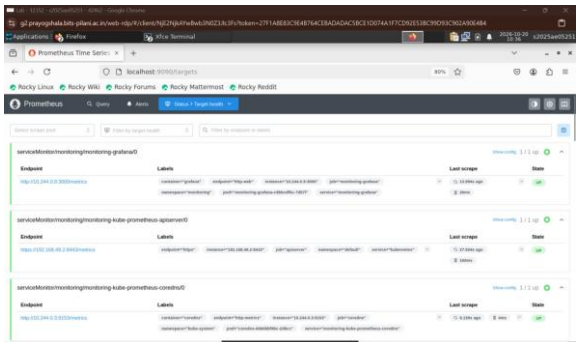


Figure 18: Prometheus targets – API endpoints in UP state

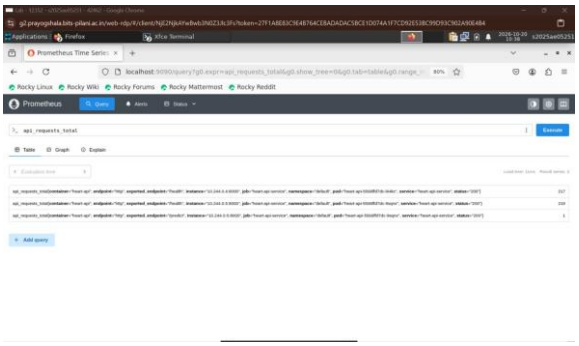


Figure 19: Prometheus query on api_requests_total

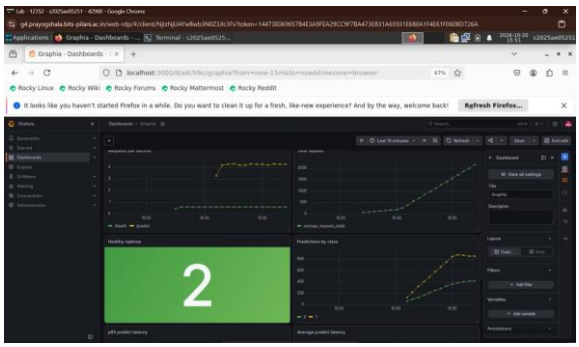


Figure 20: Grafana dashboard (request rate, totals, replicas)

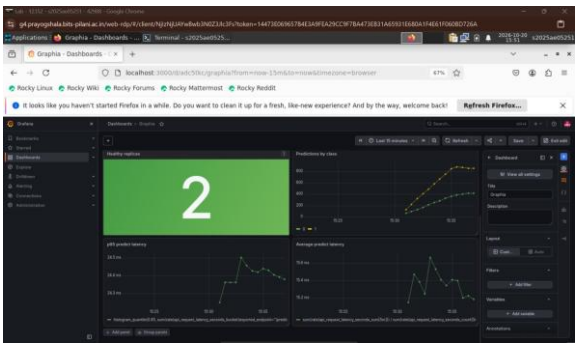


Figure 21: Grafana dashboard (prediction mix and latency)

11. System Architecture

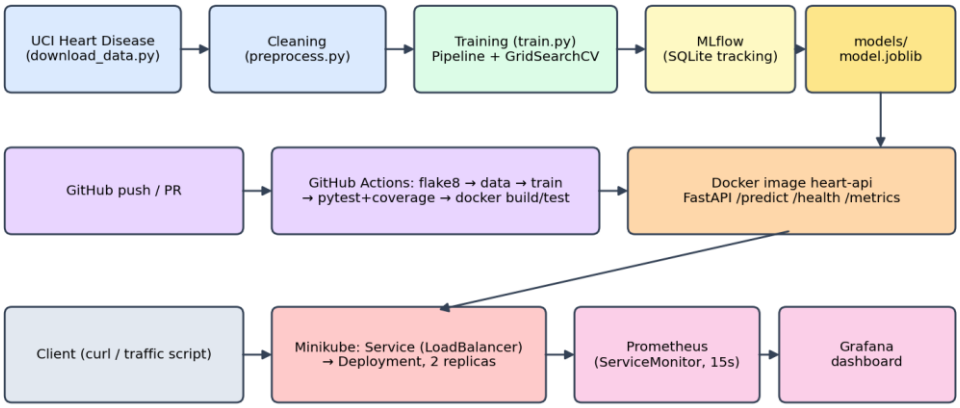


Figure 22: End-to-end architecture – data and training, CI/CD, serving and monitoring

The architecture has three loops. The training loop (top row) turns raw UCI data into a tracked, saved model. The delivery loop (middle row) uses GitHub Actions to lint, test, retrain and verify the Docker image on every change. The runtime loop (bottom row) serves the container on Kubernetes and feeds request metrics into Prometheus and Grafana.

12. Conclusions, Limitations and Future Work

The project delivers a complete, reproducible MLOps pipeline: data scripts, tuned and tracked models, a pinned environment, automated lint/test/train/docker verification, a containerised FastAPI service, a replicated Kubernetes deployment, and a monitoring stack. The final Logistic Regression pipeline reaches a hold-out ROC-AUC of 0.958 with recall of 0.929.

Limitations

- The dataset is small (303 patients, 61 in the test set), so metrics have wide uncertainty and the two models cannot be separated with confidence.
- The model is trained on a single-centre dataset and is a learning exercise, not a clinical diagnostic tool.
- Deployment is on local Minikube rather than a public cloud, so there is no public URL; Section 9 gives local access instructions.
- Monitoring covers service health and prediction mix but not data drift or model-quality monitoring.

Future work

- Add a model registry stage and automatic promotion based on metric thresholds.
- Add CD to push the image to a registry and deploy to a managed cluster (GKE/EKS/AKS) with an Ingress.
- Add drift detection on incoming features and alerting rules in Prometheus.
- Evaluate probability calibration and explainability (e.g., SHAP) for clinical trust.

Repository

Code repository: https://github.com/2025ae05251-LAB/-heart-disease-mlops